



# Módulo 7\_0

## Ecosistema Big Data – Hadoop y Spark

Curso: Data Engineer Básico

90 min

# Objetivos de Aprendizaje

---

- Describir los componentes del ecosistema Hadoop (HDFS, MapReduce, YARN) y su rol en el procesamiento distribuido
- Explicar la evolución de MapReduce a Spark, identificando las ventajas de RDDs y DataFrames
- Comparar modelos de ejecución local vs cloud para jobs Spark
- Decidir cuándo usar Big Data (Spark) vs herramientas tradicionales (Pandas/SQL)

# **1. Introducción a Hadoop**

HDFS, MapReduce, YARN

# ¿Qué es Apache Hadoop?

---

Framework de código abierto para almacenamiento y procesamiento distribuido de grandes volúmenes de datos.

# HDFS: Hadoop Distributed File System

---

# Características de HDFS

---

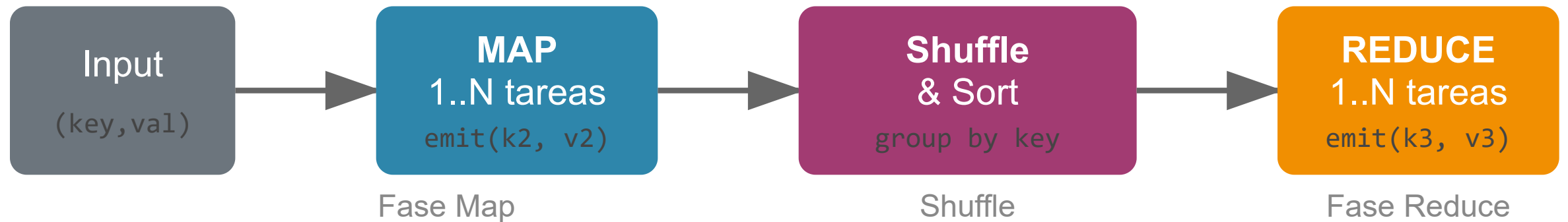
- **Alta tolerancia a fallos:** Replicación en 3 nodos por defecto
- **Archivos grandes:** Bloques de 128MB optimizados para sequential reads
- **Write-once, read-many:** No adecuado para actualizaciones frecuentes
- **NameNode:** Gestiona metadatos (namespace)
- **DataNodes:** Almacenan bloques físicamente

**En TransCore:** Almacenamiento de histórico de telemetría (100M+ registros, 2+ años)

# MapReduce: Procesamiento Paralelo

---

## Flujo de Trabajo MapReduce



Ejemplo: Contar mantenimientos por tipo

Map: (tipo\_orden, 1)

Reduce: (tipo\_orden, sum)

## Ejemplo: Contar mantenimientos por tipo

---

```
# Fase Map
def map(linea):
    orden = parse(linea)
    return (orden.tipo, 1) # (tipo_orden, 1)

# Fase Reduce
def reduce(tipo, counts):
    return (tipo, sum(counts)) # (tipo_orden, total)
```



# Limitaciones de MapReduce

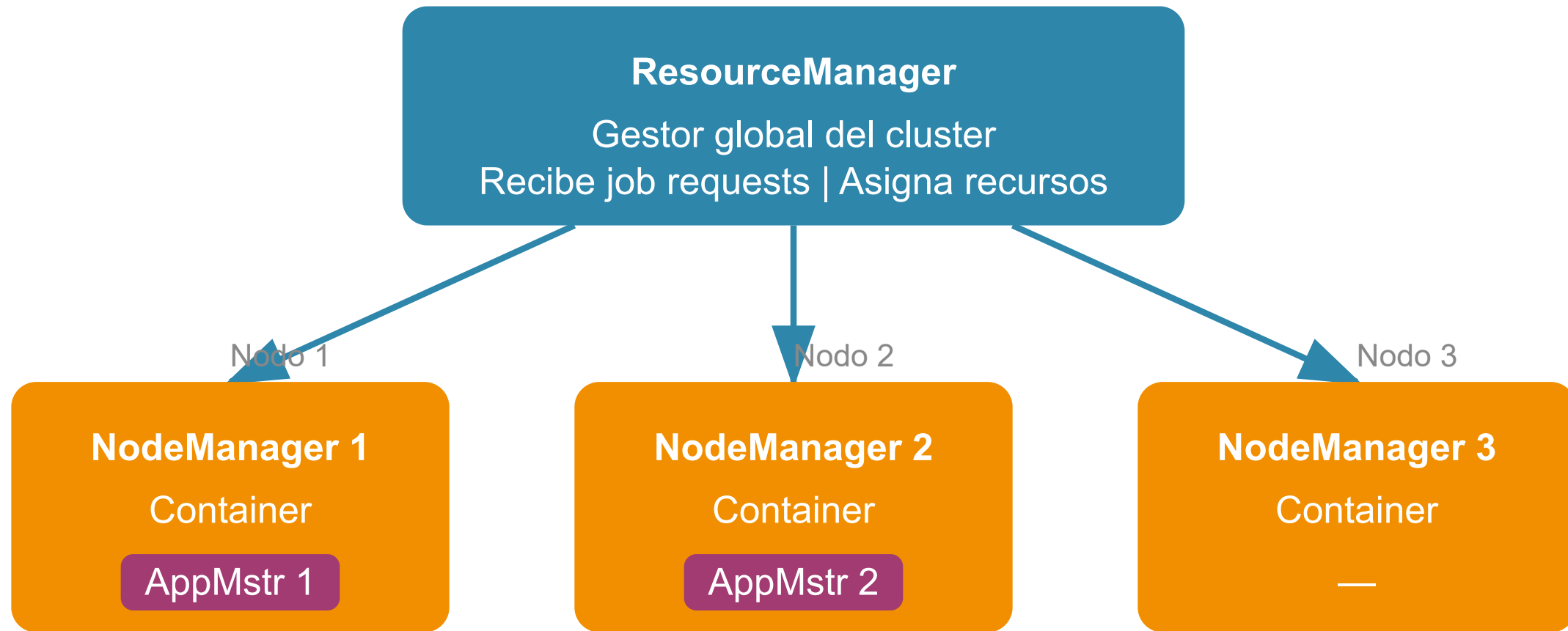
---

- **Iteraciones ineficientes:** Cada paso escribe a disco
- **Latencia alta:** No diseñado para real-time
- **Programación verbosa:** Más código que Spark

# YARN: Yet Another Resource Negotiator

---

## Arquitectura YARN



Beneficio: Múltiples motores (MapReduce, Spark, Tez) comparten el mismo cluster

# Componentes de YARN

---

- **ResourceManager:** Receives job requests, allocates resources globally
- **NodeManager:** Reports node status to ResourceManager
- **ApplicationMaster:** Negotiates resources for each job

**Beneficio:** Permite ejecutar múltiples aplicaciones (MapReduce, Spark, Tez) en el mismo cluster

## **2. Apache Spark**

El motor unificado para datos a gran escala

# Evolución: MapReduce → Spark

## Evolución: MapReduce → Spark

| Característica   | MapReduce           | Spark                       |
|------------------|---------------------|-----------------------------|
| Modelo ejecución | Disco               | Memoria (in-memory)         |
| Latencia         | Alta                | Baja                        |
| Iteraciones      | Muy costosas        | Eficientes                  |
| Streaming        | No (necesita Storm) | Sí (Spark Streaming)        |
| APIs             | Java only           | Scala, Java, Python, R, SQL |

Más moderno

Spark mantiene datos en memoria, eliminando overhead de escritura a disco

# ¿Por qué Spark?

---

- **In-memory computing:** Mantiene datos en memoria, elimina overhead de I/O a disco
- **Latencia baja:** Diseñado para procesamiento en tiempo real
- **Iteraciones eficientes:** Ideal para machine learning y agregaciones
- **Spark Streaming:** Procesamiento de streams en tiempo real
- **Múltiples APIs:** Scala, Java, Python, R, SQL

# RDDs: Resilient Distributed Datasets

---

Abstracción fundamental de Spark. Colección de elementos particionados que pueden operarse en paralelo.

## RDDs vs DataFrames

### RDDs (Resilient Distributed Datasets)

Abstracción básica de Spark

```
# Crear RDD desde archivo
telemetria = sc.textFile("s3://...")

# Transformaciones
anomalias = telemetria \
    .filter(lambda x: x[2] > 5.0) \
    .map(lambda x: (x[0], 1)) \
    .reduceByKey(lambda a,b: a+b)
```

### DataFrames

API tabular (similar a pandas)

```
# Crear DataFrame
df = spark.read.csv("s3://...",
    header=True, inferSchema=True)

# Operaciones optimizadas
kpis = df.filter(df.vibracion > 5.0)
    .groupBy("sensor_id") \
    .agg({"*": "count"})
```

DataFrames = RDDs + Optimización automática (Catalyst) + Inferencia de esquemas

# Ejemplo: RDD en PySpark

---

```
from pyspark import SparkContext

sc = SparkContext("local", "TransCoreApp")

# Crear RDD desde archivo de telemetría
telemetria = sc.textFile("s3://transcore/telemetria/2026/")

# Filtrar sensores con vibración anómala
anomalias = (
    telemetria
    .filter(lambda linea: float(linea.split(",")[2]) > 5.0)
    .map(lambda linea: (linea.split(",")[0], 1))
    .reduceByKey(lambda a, b: a + b)
)
```



# Propiedades de RDDs

---

- **Inmutable:** No se modifican, se crean nuevos
- **Tolerante a fallos:** Lineage tracking para recomputación
- **Particionado:** Datos distribuidos en el cluster

# DataFrames: API Tabular

---

API sobre RDDs, similar a pandas pero distribuido y con optimización automática.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("TransCoreAnalytics") \
    .getOrCreate()

df = spark.read.csv("s3://transcore/telemetry/2026/",
    header=True, inferSchema=True)

resultado = (
    df.filter(df.vibracion > 5.0)
    .groupBy("sensor_id")
    .agg({"*": "count", "vibracion": "avg"})
)
```

# Ventajas de DataFrames sobre RDDs

---

- **Optimización automática:** Catalyst optimizer
- **Código más legible:** Similar a pandas
- **Inferencia de esquemas:** No necesitas definir tipos
- **Integración:** Compatible con pandas y Scala

# Spark SQL: Queries sobre DataFrames

---

```
df.createOrReplaceTempView("telemetria")

resultado = spark.sql("""
    SELECT
        sensor_id,
        DATE_FORMAT(timestamp, 'yyyy-MM-dd') as fecha,
        AVG(vibracion) as vibracion_promedio,
        COUNT(*) as total_lecturas
    FROM telemetria
    WHERE vibracion > 5.0
    GROUP BY sensor_id, DATE_FORMAT(timestamp, 'yyyy-MM-dd')
    ORDER BY fecha
    """)
```

# Spark Streaming: Procesamiento en Tiempo Real

---

```
from pyspark.streaming import StreamingContext

ssc = StreamingContext(sparkContext, batchDuration=1)

# Recibir de Kafka
kafkaStream = KafkaUtils.createStream(
    ssc, "zk-server:2181", "consumer-group",
    {"telemetria": 1}
)

# Procesar cada segundo
lineas = kafkaStream.map(lambda x: x[1])
conteo = lineas.window(60, 10).count()
conteo.pprint()
```

# **3. Big Data vs Herramientas Tradicionales**

¿Cuándo usar Spark y cuándo Pandas/SQL?

# Criterios de Decisión

## Cuándo usar Big Data vs Herramientas Tradicionales

| Factor      | Pandas / SQL                     | Spark / Big Data               |
|-------------|----------------------------------|--------------------------------|
| Volumen     | < 10 GB<br>Laptop/servidor único | > 10 GB<br>Cluster distribuido |
| Latencia    | Segundos OK                      | Milisegundos                   |
| Complejidad | Consultas simples/medias         | Agregaciones complejas, ML     |
| Frecuencia  | Ad-hoc, diaria                   | Horaria, continua              |

Regla: Si los datos caben en memoria → Pandas. Si no caben → Spark/BigQuery

# Recomendación para TransCore

| Dataset                 | Volumen         | Herramienta      | Justificación          |
|-------------------------|-----------------|------------------|------------------------|
| Partes trabajo (diario) | ~2K filas       | Pandas/SQL       | Muy pequeño            |
| Órdenes mantenimiento   | ~150K activos   | Pandas/SQL       | Manejable              |
| Telemetría (2+ años)    | 100M+ registros | Spark            | Necesario              |
| Procesamiento gold KPIs | TB              | Spark + BigQuery | Agregaciones complejas |



# Regla General

---

Si los datos caben en memoria del laptop → Pandas/SQL

Si no caben o requieren procesamiento paralelo → Spark

Si son datos históricos para analytics → BigQuery (serverless)

# Lab

Comparativa ejecución local vs cloud

# Objetivo

---

Comparar rendimiento y coste de procesar 1M de registros de telemetría usando:

- **A)** Ejecución local (Pandas en laptop)
- **B)** Ejecución en cloud (Spark en EMR / Databricks)

# Escenario TransCore

---

**Dataset:** `telemetry_sample.csv` con 1M de registros

**Campos:** sensor\_id, timestamp, valor, unidad, activo\_id

**Objetivo:** Detectar sensores con promedio de vibración > 5.0

# Paso 1: Ejecución Local con Pandas (10 min)

---

```
import pandas as pd
import time

# Cargar datos
inicio = time.time()
df = pd.read_csv("telemetria_sample.csv")
print(f"Carga: {time.time() - inicio:.2f}s")

# Procesamiento: promedio por sensor
inicio = time.time()
kpis = df.groupby("sensor_id").agg({
    "valor": ["mean", "count"]
}).reset_index()
kpis.columns = ["sensor_id", "valor_promedio", "total_lecturas"]
sensores_alerta = kpis[kpis["valor_promedio"] > 5.0]
print(f"Procesamiento: {time.time() - inicio:.2f}s")
```

## Resultados Esperados: Pandas Local

---

- Tiempo carga: ~5-10s para 1M de registros
- Tiempo procesamiento: ~2-5s
- Memoria consumida: ~500MB

## Paso 2: Ejecución Cloud con Spark (10 min)

---

```
# Configuración Databricks Community (gratuito)
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("TransCore-Telemetry") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()

# Cargar datos desde S3
df = spark.read.csv(
    "s3://transcore-datalake/telemetry/telemetry_sample.csv",
    header=True, inferSchema=True
)
df.cache()

# Procesamiento distribuido
kpis = df.groupBy("sensor_id").agg(
    sparkavg("valor").alias("valor_promedio"),
    spark_count("*").alias("total_lecturas")
)
```

## Resultados Esperados: Spark Cloud

---

- Tiempo setup cluster: ~3-5 min (cold start)
- Tiempo procesamiento: ~30-60s (datos en memoria)
- Coste DBU: ~\$0.50-1.00 por ejecución



# Análisis Comparativo

| Aspecto         | Local (Pandas)   | Cloud (Spark)          |
|-----------------|------------------|------------------------|
| Tiempo total    | ~15s             | ~5-7 min (incl. setup) |
| Infraestructura | Laptop propio    | Cluster temporal       |
| Coste directo   | \$0 (amortizado) | \$0.50-1.00/ejecución  |
| Escalabilidad   | Limitada (RAM)   | Horizontal (n nodos)   |
| Mantenimiento   | Bajo             | Medio                  |

## Conclusión para TransCore

---

- **Procesamiento ad-hoc** de datasets pequeños → Pandas local
- **Pipelines programados** sobre datos grandes → Spark en EMR/Databricks
- **Analytics interactivo** sobre datos históricos → BigQuery

# Resumen

# Puntos Clave

---

- **HDFS:** Almacenamiento distribuido con replicación; optimizado para archivos grandes
- **MapReduce:** Procesamiento paralelo en dos fases (Map/Reduce); alto overhead por I/O a disco
- **YARN:** Gestor de recursos que permite múltiples motores en el mismo cluster
- **RDDs:** Abstracción básica de Spark; control de bajo nivel, mayor flexibilidad
- **DataFrames:** API tabular sobre RDDs; optimizado automáticamente por Catalyst
- **Cuándo Spark:** Datos > 10GB, agregaciones complejas, procesamiento distribuido
- **Cuándo Pandas:** Datos pequeños, exploración ad-hoc, baja latencia aceptable

# Referencias

# Recursos Adicionales

---

- [Apache Spark Documentation](#)
- [Hadoop Documentation](#)
- [Databricks Community Edition](#) (free cluster)
- [AWS EMR Getting Started](#)
- [PySpark Tutorial](#)



**Gracias!**  
**Módulo 7\_0 Completado**

Siguiente: Módulo 7 - Spark Advanced